

Desafio 01: Onboarding SecOps & Soberania Técnica (Fase 1)

Objetivo: Estabelecer a infraestrutura base individual e a esteira de segurança (Security Gate), garantindo que cada engenheiro esteja apto a integrar os squads da PCDF com um ambiente padronizado e seguro.

1. Contexto Estratégico

Alinhado ao **Domínio Oito do CISSP (Software Development Security)**, estabelecemos que a segurança não é um estágio final, mas um atributo intrínseco que deve ser **incorporado desde a primeira linha de código**. Este domínio preconiza que a proteção de ativos digitais exige a aplicação de controles de segurança em todo o Ciclo de Vida de Desenvolvimento de Software (SDLC), mitigando vulnerabilidades antes que o software entre em produção.

Nesta disciplina, operamos sob a abordagem **AI-First**, utilizando aceleradores de produtividade como o **GitHub Copilot**. Entretanto, essa agilidade é equilibrada pelo rigor da **Soberania Técnica**. Sob a ótica do CISSP, isso significa implementar o princípio da **Defesa em Profundidade**: embora a IA auxilie na geração de código, a responsabilidade final pela auditoria e integridade da solução é do engenheiro.

Nossa arquitetura deve ser inerentemente resiliente, projetada para operar em **ambientes controlados e soberanos (on-premise)**, atendendo aos requisitos críticos da **PCDF**. Isso envolve a aplicação prática de conceitos como:

- **Fail-Safe (Falha Segura):** Garantir que, em caso de erro no código gerado por IA, o sistema não exponha dados sensíveis ou permissões administrativas.
- **Sandboxing e Isolamento:** O uso mandatório de **Docker** para garantir que processos de IA rodem de forma estanque, protegendo o núcleo do sistema operacional.
- **Auditoria Contínua (SAST):** A integração de ferramentas como **Bandit** e **Pydantic** para validar a conformidade do código com padrões de segurança governamentais, garantindo que a inovação tecnológica nunca comprometa a custódia da prova digital.

2. Setup Mandatório do Ambiente (Localhost)

Para garantir a paridade com a infraestrutura da PCDF, cada aluno deve configurar:

- **Python 3.12:** Versão estável obrigatória (evitar versões experimentais como 3.14).
- **Poetry:** Gestor de dependências oficial. Não utilizaremos `pip` puro ou `requirements.txt`. O controle deve ser via `pyproject.toml`.
- **Django 5.0+:** Framework base para os 4 aplicativos da PCDF.
- **Docker:** Para isolamento de serviços e futura orquestração de modelos de IA locais (Ollama/Llama 3).

3. Esteira de Segurança (DevSecOps)

Seu repositório individual deve conter um workflow de **GitHub Actions** configurado para executar:

- **SAST (Bandit):** Auditoria estática para detectar funções perigosas.
 - **SCA (Safety):** Scan de vulnerabilidades em bibliotecas de terceiros.
 - **Linter/Formatter (Ruff):** Garantia de qualidade e padronização de código.
 - **Secret Scanning:** Proteção contra exposição de chaves e tokens.
-

O Produto Final: Relatório de Prontidão Técnica

Ao concluir as atividades no seu **repositório próprio**, você deverá elaborar um breve relatório (Markdown ou PDF, mais ou menos como no apêndice¹, ao final desse documento) confirmando os seguintes marcos:

1. **Evidência de Ambiente:** Print ou log do comando `poetry env info` confirmando o uso do Python 3.12.
2. **Evidência de Segurança:** Relatório do Bandit com **zero issues** identificado no código inicial.
3. **Evidência de CI:** Badge ou log do GitHub Actions confirmando que o pipeline de DevSecOps passou com sucesso (Build: Passing).
4. **Declaração de Prontidão:** Confirmação de que o código está pronto para migração via *Git Remote* para a organização oficial.

4. Migração para a Organização **rmchaimalm**

O processo de migração para o ecossistema oficial da disciplina será iniciado assim que a coordenação finalizar a liberação das ferramentas avançadas por meio do **GitHub Global Campus for Teachers**. Este marco é fundamental para a transição da Fase de Iniciação para a Fase de execução em escala industrial.

O programa **GitHub Education** viabilizará o acesso a recursos profissionais de alta performance — incluindo **GitHub Pro**, **GitHub Copilot** e **GitHub Advanced Security** — essenciais para sustentar o rigor técnico exigido pelos quatro aplicativos da PCDF. A utilização desses aceleradores será acompanhada por auditorias rigorosas, garantindo que a produtividade assistida por IA não comprometa a **Soberania Técnica** nem a integridade do código.

Assim que a organização **rmchaimalm** for oficialmente provisionada, serão fornecidas as instruções detalhadas para a configuração do `git remote add upstream`. Esta estratégia de versionamento foi desenhada para garantir a persistência e a rastreabilidade de todo o

histórico de trabalho realizado durante o mês de março, permitindo que a fundação de **DevSecOps** estabelecida nos repositórios individuais seja integrada de forma transparente ao ambiente de produção.

Essa migração marca o alinhamento definitivo com o **Domínio 8 do CISSP**, onde o controle de configuração e a gestão de mudanças tornam-se ativos críticos para a garantia da **Cadeia de Custódia Digital** e a entrega de valor real aos nossos *stakeholders*

5. Checkpoint de Prontidão Técnica: Comandos e Prazos

Para consolidar a sua **Fase 1 (Iniciação e Soberania Técnica)**, utilize o roteiro abaixo. Estes passos garantem que seu ambiente individual atenda aos critérios de **Defesa em Profundidade** e **Auditoria Estática (SAST)** exigidos para os aplicativos críticos da PCDF:

Resumo de Comandos para o Checklist:

1. Gestão de Dependências e Isolamento (Zero Trust):

Bash

Configurar o Poetry para criar o ambiente virtual (.venv) dentro da pasta do projeto

poetry config virtualenvs.in-project true

Inicializar o projeto garantindo o determinismo via pyproject.toml

poetry init

Instalar a stack de segurança, validação e qualidade (DevSecOps)

poetry add --group dev bandit ruff safety pydantic

2. Protocolo de Auditoria e Qualidade (Domínio 8 CISSP):

Bash

Executar scanner de vulnerabilidades lógicas (SAST)

poetry run bandit -r .

Executar linter e formatter (Ruff) para garantir conformidade e código limpo

poetry run ruff check .

Validar integridade e CVEs em bibliotecas de terceiros (SCA)

poetry run safety check



Calendário de Marcos (Milestones):

- **Prazo para conclusão do ambiente e entrega do Relatório de Prontidão: Até 30/03/2026.** Via aprender, em
 - **Nota:** A entrega deste relatório em seu repositório próprio (via arquivo `README.md` ou `PDF`) contendo os logs de sucesso dos comandos acima é **condição obrigatória** para a migração ao ecossistema industrial da disciplina.
- **Primeira Aula Presencial (Kickoff de Sprints e Formação de Squads): 01/04/2026.**
 - Neste marco, iniciaremos a integração dos repositórios individuais à organização